

Merkle-Damgård hash function

Introduction

Imagine sending a file to your friend's device in the other side of the planet. The device received the file, but now there is one tiny issue. How does he know if the file has been tampered with? Usually, sending a file into the communication channel required encryption algorithms to encrypt the file so that it remains confidential. What about its integrity? That is the job of hash function. This report will explore more about the Merkle-Damgård hash function, test it against SHA-256, the most widely used hash function currently, with various attacks.

Conclusion

Hash function is the crucial component for verifying the integrity of any internet packages. Yet not every hash function is flawless. Hash functions that are based on the Merkle-Damgård construction are not completely collision resistance and they are vulnerable to length extension attack. Additionally, multiple tests were performed on two hash functions, toyhash and SHA-256, where both are based on the Merkle-Damgård construction. The results indicated that SHA-256 outperformed toyhash when it comes to throughput.

Technical background

What is a hash function? [2]

Hashing in the world of IT is a method that convert an input into a fixed size and unique output. It is used in many different ways such as searching for data or storing password. As mentioned above, hash function is used as a part of integrity verification of a file after transportation. The fixed size and unique output is known as a hash value. It works as a fingerprint for the data and thus, it is used to check if the data have been changed before its arrival. A strong hash function needs to have these two properties, one-way and collision resistance. One-way property means that an input should only give a single hash value. Collision resistance property means that it should not be possible for two distinct inputs to give the same hash value.

Avalanche effect

As mentioned above, one of the properties of a hash function is one-way. Due to that, a single change in the input will give a completely different hash value. This is known as the avalanche effect.

Merkle-Damgård construction [1]

Merkle-Damgård construction is a hash framework used in MD5, SHA-1 and SHA-2. The construction consists of a compression function and an initialisation vector, which a fixed string. Furthermore, it is also a block-based hash function. In another word, the input will be divided into evenly sized blocks and therefore, the input has to be padded with bits if the size of the input is not

divisible by the block size. Additionally, the final block is also encoded with the length of the original message. The algorithm starts with the initial block being compressed together with the initialisation vector, which gives the first hash value. The hash value is thereafter used to compress together with the next block, which gives the second hash value. It continues until the final hash value is achieved.

Merkle tree [4]

This is a tree structure that consist of two parts, non-leaf nodes and leaf nodes. Leaf nodes contain the actual data and non-leaf nodes contain the hash value of their respective child nodes. At the top is the root node, which contains the hash value of all its child nodes. Therefore, the root node acts as a fingerprint the data set. Non-leaf nodes (except for root note) are concatenated and hashed, and that produces a node for the next level in the tree. Performing look up is quick because it is a tree and furthermore, it can support a large data set. Verifying the data does not need the entire data set, but only a list of the data's associative nodes, the data block and the root node.

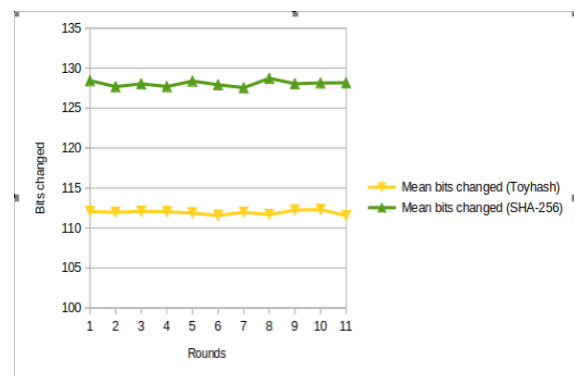
Message authentication code [5]

MAC is the hash value of the concatenation between a secret key and a message. This is used to verify the authenticity of the message when sending through the communication channel. The arrived package will contain the encrypted message and the MAC. The MAC get extracted, the message gets decrypted, and together they get hashed. The hash output is then compared with the MAC that was in the package.

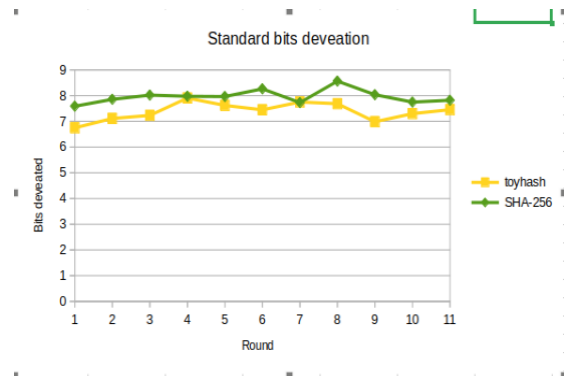
Tests and results

Avalanche effect experiment

Round	Trials	Mean bits changed (Toyhash)	Mean bits changed (SHA-256)
1	200	112,065	128,425
2	225	111,964444444444	127,684444444444
3	250	112,096	128,044
4	275	112,036363636364	127,698181818182
5	300	111,873333333333	128,376666666667
6	325	111,569230769231	127,92
7	350	111,96	127,548571428571
8	375	111,688	128,722666666667
9	400	112,2425	128,05
10	425	112,315294117647	128,145882352941
11	450	111,546666666667	128,166666666667

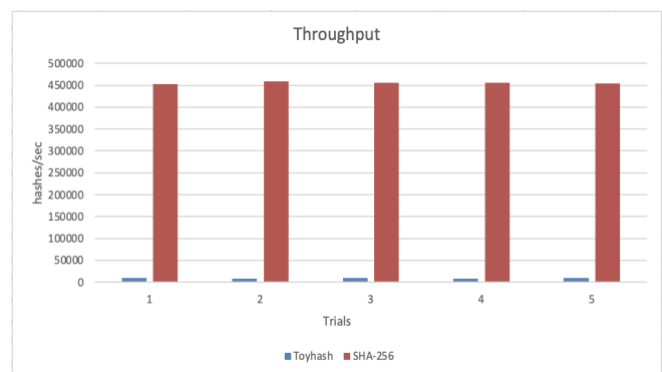


Round	Trials	toyhash	SHA-256
1	200	6,74839054886419	7,58777800149665
2	225	7,11234156021334	7,85665904555015
3	250	7,23151325795644	8,0201037399772
4	275	7,90710528888969	7,9749492083669
5	300	7,61953337737221	7,96459596520071
6	325	7,4469802465764	8,26185302366329
7	350	7,75102573857178	7,72910534200132
8	375	7,6848762297558	8,56335718174949
9	400	6,98489038926159	8,03601269287197
10	425	7,2992748371869	7,74732671831235
11	450	7,45567509518886	7,81970445591008



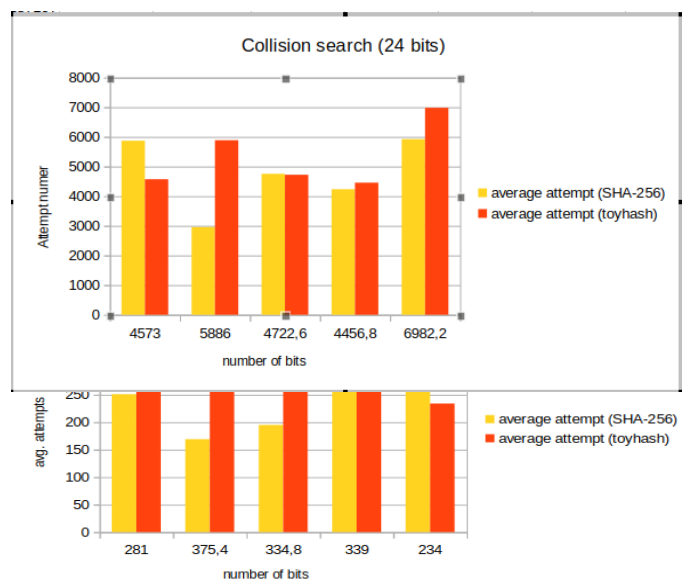
Throughput experiment

Trial	Toyhash	SHA-256
1	9534	453059
2	9388	458715
3	9527	455215
4	9476	455259
5	9559	453280



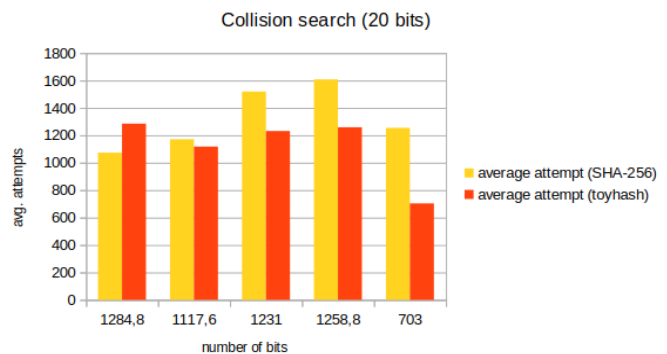
Collision search experiment

Trial	average attempt (toyhash)	average attempt (SHA-256)
1	4573	5870
2	5886	2957,6
3	4722,6	4752,8
4	4456,8	4233,4
5	6982,2	5925,4



Trial	average attempt (toyhash)	average attempt (SHA-256)
1	281	251
2	375,4	169,2
3	334,8	195,2
4	339	371,6
5	234	349,2

Trial	average attempt (toyhash)	average attempt (SHA-256)
1	1284,8	1073,4
2	1117,6	1170,8
3	1231	1518,8
4	1258,8	1607,2
5	703	1254,4



Tasks

Task1:

- 1) To mask a value to 32 bits basically means to set a variable size to only 32 bits. This is necessary in python because a python variable can exceed 32 bits limit.
- 2) Bit shift manipulates the bits of a variable by moving the 1 and 0s further to the left or the right side. Bit rotations are similar to bit shift, but the only difference is when the bits move beyond the limit, those bits appear on the other end. Rotation is important because of how sensitive a hash is. If a bit is loss, the hash will be unusable (avalanche effect).

Task2:

- 1) Compression function is used to compress the input into a fixed size output.
- 2) The combination of addition, XOR and rotation makes it hard to find a specific hash value with inputs other than the original input. This is the one-way property of a hash function

Task3:

- 1) Padding is necessary for dividing a given message to even-sized block.
- 2) Message length is necessary because otherwise it would be easy for attacker to perform length extension attack.
- 3) Preimage resistance is a property of a hash function that makes it difficult to compute the original input by using a hash function on a variable.

Task4:

- 1) A well-designed hash function will give a completely different hash value even if the input has been changed slightly
- 2) It seems like the number of bits changed are higher for SHA-256 than toyhash. This makes sense since SHA-256 produces a 256-byte hash while toyhash only produces 32-byte hash.

Task5:

- 1) In the context of hash function, an attacker can find the collided value, that is the hash value produced by two distinct inputs, by using a large enough group of inputs and sort the result. Consider a hash function that produces an n -bits hash value. The hash function is collision resistance up to $2^{n/2}$ inputs. [5]
- 2) The average attempt for finding collision increases as the output size got bigger. It is a little bit unexpected how the growth in average attempts is because I expected the average attempts would be somewhat stabilized once the right number of attempts is past $2^{n/2}$ (*500 000 was set*). Another noticeable result is that the average attempts for both SHA-256 and toyhash are about the same.

Task6:

- 1) This is a method of attack that forges a MAC without the secret key. It works by taking the final hash output and concatenate extra lines (*e.g a database query*) to achieve certain goals. This attack exploits the vulnerability of the Merkle-Damgård construction where the final hash output can be used to resume hashing.
- 2) All Eve needs is a copy of the MAC and the length of the secret key. Given those prerequisites, Eve concatenates the hash value of her desired data at the end. With that done, Eve has forged a MAC with the inclusion of her data.
- 3) One way is to use a hash function that does not utilise Merkle-Damgård construction such as, SHA-3. Another way is to hash the MAC again with the secret key (HMAC).

Task7:

- 1) Merkle tree is useful to protect the integrity of a large data set because it does not need to go through the entire data set to verify a single data block, but only a subset of the nodes (leaf node and non-leaf node)
- 2) The inclusion proof needs to contain the hash value of the sibling node from each level.

Task8:

- 1) The biggest difference is in the avalanche effect experiment where toyhash's mean bits changes are significantly lesser than SHA-256
- 2) Based on the results, toyhash seems to perform worse than SHA-256 on avalanche effect experiment. However, this is expected because SHA-256 produces a 256-byte hash. On collision resistance, the results are about the same. Throughput is where toyhash performed the weakest. It was clear that SHA-256 produced way more hashes than toyhash did.

Source

- 1) Tamassia, Roberto & Goodrich, Michael. (2013). Introduction to computer security (1st edition). Introduction, chapter 3.5, Cryptography, chapter 3.1. Pearson international content.
- 2) Tamassia, Roberto & Goodrich, Michael. (2013). Introduction to computer security (1st edition). Introduction, chapter 3.5. Pearson international content.
- 3) Tamassia, Roberto & Goodrich, Michael. (2013). Introduction to computer security (1st edition). Cryptography, chapter 3.2. Pearson international content.

- 4) Curran, Brian. (2023, 6. november). What is a Merkle Tree? Beginner's guide to this blockchain component. <https://blockonomi.com/merkle-tree/>
- 5) Tamassia, Roberto & Goodrich, Michael. (2013). Introduction to computer security (1st edition). Cryptography, chapter 3.4. Pearson international content.